Pearls AQI Predictor

A Serverless Machine Learning Pipeline for 3-Day Air Quality Forecasting in Karachi

Data Science Internship Project Report
Prepared by: Wania Khan
Jinnah University for Women, Karachi — BS Computer Science

1. Project Overview

This project implements an end-to-end, serverless machine learning pipeline that forecasts Karachi's Air Quality Index (AQI) up to three days ahead. It covers the full lifecycle of a production ML system: automated data collection, feature engineering, model training and evaluation, deployment automation, and an interactive dashboard with explainability.

1.1 Objectives

- Collect live and historical air-quality and weather data for Karachi automatically.
- Engineer time-based and derived features (lag, rolling average, AQI change rate).
- Train and evaluate forecasting models for 24h, 48h, and 72h horizons using RMSE, MAE, and R^2.
- Automate the pipeline end-to-end using GitHub Actions (hourly feature updates, daily retraining).
- Present results through an interactive dashboard with model explainability (SHAP).

1.2 Technology Stack

Category	Tools Used
Language	Python 3.11
ML Libraries	Scikit-learn (Ridge Regression, Random Forest)
Feature Store / Model Registry	Hopsworks
Automation / CI-CD	GitHub Actions
Dashboard	Streamlit
Explainability	SHAP
Data Sources	AQICN API (live), OpenWeather API (historical pollutants), Open-Meteo API (historical weathe
Version Control	Git / GitHub

2. System Architecture

The system follows a five-stage serverless architecture, with each stage automated to run independently on a schedule:

- **Feature Pipeline** — runs hourly via GitHub Actions. Fetches live AQI and weather readings for Karachi from the AQICN API, engineers time-based features (hour, day, month, day of week), and writes them to the Hopsworks feature store.
- **Historical Backfill** — a one-time pipeline that populated ~30 days of hourly training data using OpenWeather (pollutants) and Open-Meteo (weather), since AQICN's free tier only exposes real-time data.

- **Training Pipeline** — runs daily via GitHub Actions. Pulls all available features, engineers lag/rolling/change-rate features, trains separate Ridge Regression and Random Forest models for each forecast horizon (24h, 48h, 72h), and registers the best-performing model per horizon in the Hopsworks Model Registry.
- **Dashboard** — a Streamlit application that loads the latest features and the three registered models, displaying the current AQI, the 3-day forecast, a historical trend chart with US EPA AQI reference bands, and SHAP-based feature importance.
- **Automation** — GitHub Actions workflows (feature_pipeline.yml, training_pipeline.yml) run the above on schedule using repository secrets for credentials, requiring no manual intervention.

3. Methodology

3.1 Feature Engineering

Raw pollutant and weather readings (PM2.5, PM10, O■, NO■, SO■, CO, temperature, humidity, pressure, wind speed) are combined with time-based features (hour, day, month, day of week) and derived features: a 1-hour AQI lag, a 3-hour rolling AQI average, and an AQI change rate. These capture short-term pollution trends that raw readings alone do not.

3.2 Forecast Framing

Each model is trained to predict AQI a fixed number of hours into the future (24h, 48h, or 72h), rather than the AQI at the same timestamp as its inputs. Data is split chronologically (not randomly) into training and test sets, since shuffling would leak future information into the training set for a forecasting problem.

3.3 Data Quality Handling

The live AQICN station used for real-time data only reports PM2.5, not the other pollutants. Rather than imputing the missing pollutants with a running median — which was found to degrade model quality over time as more incomplete live rows accumulated — the training pipeline excludes rows missing real pollutant readings, keeping training data quality stable regardless of how much live data accumulates.

4. Challenges and Solutions

Building and operating this pipeline surfaced a number of real engineering issues, each diagnosed and resolved during development:

Challenge	Resolution
Hopsworks' Windows dependency (twofish) required a C++ compiler not available by default on Windows.	Created a dedicated [illegible] and installed the dependency via c
Hopsworks writes certificates to a Unix-style /tmp path that does not exist on Windows.	Manually created a C:\tmp directory to satisfy the path requirement.
Initial model achieved artificially high R^2 (0.99) due to target leakage — AQI was computed directly from PM2.5 [illegible]	Removed PM2.5 from the model's input features and [illegible] for historical data, and ge
OpenWeather's free tier provides historical pollutants but not historical weather.	Used Open-Meteo's free historical weather archive (no API key required) to back

A code update adding weather features to the training script was used locally but never pushed to GitHub, causing 12+ days of	Diagnosed by directly inspecting the Model Registry's version history and compa
Dashboard model loading picked the wrong model version once version numbers reached double digits, due to	Fixed the comparison to cast version numbers to integers before selecting the la

5. Results and Evaluation

Models were evaluated on a chronological hold-out test set using Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and R^2 (coefficient of determination). Results below reflect the current production models, trained on real pollutant and weather data with the leakage issue corrected.

Horizon	Best Model	RMSE	MAE	R^2
24 hours	Ridge Regression	4.99	4.35	-0.34
48 hours	Random Forest	8.95	7.80	-4.90
72 hours	Random Forest	8.48	7.58	-5.32

Note: negative R^2 values indicate the models currently perform below a naive baseline that simply predicts the historical average AQI. This is reported transparently as a genuine finding rather than a favorable but misleading metric.

5.1 Interpretation

The negative R^2 values are attributable primarily to the limited size of the training dataset (approximately one month of hourly data for a single city) rather than a flaw in the modeling approach. Forecasting AQI multiple days ahead is a genuinely difficult problem even with extensive data, since it depends on evolving atmospheric conditions not fully captured by a single station's readings. RMSE remains in a usable range (roughly 5–9 AQI points), meaning the point forecasts are reasonably close even where the model does not yet outperform a naive average.

5.2 Explainability

SHAP (SHapley Additive exPlanations) values were computed for the 24-hour model and are surfaced in the dashboard as a feature-importance chart. This satisfies the project's requirement for model explainability and allows a viewer to see which inputs — pollutant readings, weather, time of day, or recent AQI trend — are driving a given forecast.

6. Limitations and Future Work

- **Data volume** — training data currently spans roughly one month. Continued hourly collection via the automated pipeline will steadily improve this.
- **Single monitoring station** — the live AQICN station used covers only one location in Karachi and reports PM2.5 only; incorporating additional stations would improve coverage and pollutant completeness.
- **Station reliability** — the live station was observed to report a frozen (stale) reading for an extended period, which the pipeline recorded without producing an alert. Adding a staleness check for future work.
- **Deep learning models** — the project specification allows for TensorFlow/PyTorch models; given the current data volume, tree-based and linear models were prioritized. A recurrent or sequence model could be revisited once more data has accumulated.

7. Conclusion

This project delivered a complete, automated, serverless AQI forecasting system: hourly data collection, a historical backfill, multi-horizon forecasting models with proper evaluation, full CI/CD automation via GitHub Actions, an interactive dashboard, and model explainability via SHAP. Beyond the modeling itself, the project involved substantial applied engineering — diagnosing environment issues, a data leakage bug, and a silent deployment gap — all of which are documented above as part of the development record.

8. Project Links

GitHub Repository: https://github.com/waniakhan6/pearls-aqi-predictor